

Sending HTTP Requests in React

Fetching, Posting & Deleting Data — in Plain English

A beginner-friendly visual guide with examples for every concept

1. How To Connect To A Database?

React (running in the browser) **cannot directly talk to a database** like MySQL or MongoDB. That would be dangerous — anyone could see your passwords! Instead, React talks to a **Backend / API**, and the backend talks to the database.

React App
(Browser)

■ HTTP
Request ■

Backend / API
(Server)

■ Query ■

Database (MySQL,
MongoDB...)

Key Rule: Never put database passwords or secret keys in your React code. Always use a backend API as the middleman. React only ever sees the API's response.

2. How To Send HTTP Requests?

An **HTTP Request** is a message your React app sends to a server asking for data or telling it to do something. There are two main tools in React for this: **fetch()** (built into browsers) and **axios** (a popular library).

HTTP Method	What it does	Real-world analogy
GET	Read / fetch data from server	Asking a librarian for a book
POST	Send new data to server (create)	Filling out a form and submitting it
PUT	Replace existing data fully (update)	Rewriting an entire page of notes
PATCH	Update part of existing data	Fixing just one word in your notes
DELETE	Remove data from server	Tearing a page out of a notebook

3. Sending a GET Request via useEffect

You usually want to **fetch data when your component first loads**. The best place for this is inside **useEffect** with an empty dependency array `[]`, which means: run this only once, when the component appears on screen.

Basic GET request example:

```
import { useState, useEffect } from 'react';

function UserList() {
  const [users, setUsers] = useState([]);
  useEffect(() => {
    // fetch() sends a GET request by default
    fetch('https://jsonplaceholder.typicode.com/users')
      .then(response => response.json()) // Convert response to JSON
      .then(data => setUsers(data)); // Save data in state
  }, []); // Empty [] = run only once on mount
  return (
    {users.map(user => {user.name})}
  );
}
```

4. Using async / await

async/await is a cleaner way to write code that waits for something (like a network request) to finish. Instead of chaining `.then()` calls, you write code that **looks like it runs top-to-bottom** — much easier to read.

With `.then()` chains

```
useEffect(() => {
  fetch('/api/users')
    .then(res => res.json())
    .then(data => {
      setUsers(data);
    })
    .catch(err => {
      console.error(err);
    });
}, []);
```

With `async / await` (cleaner!)

```
useEffect(() => {
  async function fetchData() {
    try {
      const res = await fetch('/api/users');
      const data = await res.json();
      setUsers(data);
    } catch (err) {
      console.error(err);
    }
  }
  fetchData();
}, []);
```

Important: You cannot make `useEffect`'s callback itself `async`. Instead, define an `async` function inside `useEffect` and then call it — just like the example on the right above.

5. Handling Loading States

Network requests take time. While waiting, you should show a **loading indicator** so users know something is happening — not a blank screen. Use a simple **isLoading** state boolean for this.

```
function UserList() {
  const [users, setUsers] = useState([]);
  const [isLoading, setIsLoading] = useState(false); // Track loading
  useEffect(() => {
    async function fetchUsers() {
      setIsLoading(true); // Show spinner BEFORE fetching
      const res = await fetch('/api/users');
      const data = await res.json();
      setUsers(data);
      setIsLoading(false); // Hide spinner AFTER data arrives
    }
    fetchUsers();
  }, []);
  if (isLoading) return Loading... please wait;
  return {users.map(u => {u.name})};
}
```

6. Importing from Error.jsx & Handling HTTP Errors

Things go wrong — the server might be down, or the URL might be wrong. You should always handle errors gracefully. A great pattern is to create a reusable **Error.jsx** component and import it wherever needed.

Step 1 — Create Error.jsx (reusable error component):

```
// Error.jsx
export default function ErrorMessage({ title, message }) {
  return (
    {title}
    {message}
  );
}
```

Step 2 — Use it in your component with proper error handling:

```
import ErrorMessage from './Error.jsx';

function UserList() {
  const [users, setUsers] = useState([]);
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState(null);
  useEffect(() => {
```

```

async function fetchUsers() {
  setIsLoading(true);
  try {
    const res = await fetch('/api/users');
    // fetch() does NOT throw on 4xx/5xx – check manually!
    if (!res.ok) {
      throw new Error('Failed to fetch users. Status: ' + res.status);
    }
    const data = await res.json();
    setUsers(data);
  } catch (err) {
    setError({ title: 'Error!', message: err.message });
  }
  setIsLoading(false);
}
fetchUsers();
}, []);

if (isLoading) return Loading...;
if (error) return ;
return {users.map(u => {u.name})};
}

```

Key gotcha: `fetch()` only throws an error for **network failures** (no internet). A 404 (Not Found) or 500 (Server Error) response does NOT throw — you must check **response.ok** yourself and throw manually.

7. Transforming Fetched Data

The data you get from an API often doesn't look exactly how you want it. You can **transform (reshape) the data** before storing it in state — rename keys, filter items, combine fields, etc.

Example — API gives ugly keys, we clean them up:

```
// Raw API response looks like this:
// { usr_nm: 'Alice', usr_age: 30, usr_email: 'alice@test.com' }

const data = await res.json();

// Transform: rename keys to friendly names
const cleanData = data.map(user => ({
  name: user.usr_nm,
  age: user.usr_age,
  email: user.usr_email,
}));

setUsers(cleanData); // Now stored in a clean format
```

Transformation	How to do it	When to use
Rename keys	.map() with new object	API uses ugly or abbreviated names
Filter items	.filter()	You only want some items
Sort items	.sort()	API doesn't sort for you
Combine fields	Template literals in .map()	e.g. first + last name
Pick only needed	Destructure in .map()	API sends too much data

8. Extracting Code & Improving Code Structure

As your app grows, putting all fetch logic inside every component gets messy. The solution: **extract HTTP logic into a separate file** (like http.js). This keeps components clean and makes code reusable.

■ Messy — everything in component

Component handles UI + fetching
+ error handling + data transform

Hard to reuse
Hard to test
Gets very long

■ Clean — logic in http.js

http.js: all fetch functions
Component: only UI logic

Easy to reuse anywhere
Easy to test separately
Short and readable

http.js — your centralized fetch file:

```
// http.js — all API calls live here
const BASE_URL = 'https://api.example.com';

export async function fetchUsers() {
  const res = await fetch(`${BASE_URL}/users`);
  if (!res.ok) throw new Error('Could not fetch users');
  return res.json();
}

export async function createUser(userData) {
  const res = await fetch(`${BASE_URL}/users`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(userData),
  });
  if (!res.ok) throw new Error('Could not create user');
  return res.json();
}
```

Using it in a component — super clean:

```
import { fetchUsers } from './http.js';

useEffect(() => {
  async function load() {
    const data = await fetchUsers(); // One clean line!
    setUsers(data);
  }
  load();
}, []);
```

9. Sending Data with POST Requests

A **POST request** is used to **send new data to a server** — like creating a new user, submitting a form, or adding an item. Unlike GET, you attach the data in the **request body**.

POST request — add a new user:

```
async function addUser(newUser) {
  const response = await fetch('https://api.example.com/users', {
    method: 'POST', // Tell server this is a POST
    headers: {
      'Content-Type': 'application/json', // Tell server we're sending JSON
    },
    body: JSON.stringify(newUser), // Convert JS object to JSON string
  });
  if (!response.ok) {
    throw new Error('Failed to add user');
  }
}
```

```

const savedUser = await response.json(); // Server returns the saved user
return savedUser;
}

// Call it on button click:
async function handleSubmit() {
const created = await addUser({ name: 'Alice', email: 'a@b.com' });
setUsers(prev => [...prev, created]); // Add to existing list
}

```

Part	What it does
<code>method: 'POST'</code>	Tells server you are creating new data
<code>Content-Type: application/json</code>	Tells server the body is JSON format
<code>JSON.stringify(data)</code>	Converts your JS object to a JSON text string
<code>await response.json()</code>	Reads the server's response (usually the created item)

10. Using Optimistic Updating

Optimistic Updating means you **update the UI immediately** (before the server confirms), assuming the request will succeed. If it fails, you **roll back** to the old state. This makes your app feel instant and snappy!

■ Normal (pessimistic)	■ Optimistic
1. User clicks 2. Wait for server... 3. Server replies 4. Update UI	1. User clicks 2. Update UI instantly 3. Send request to server 4. If error: rollback UI
Feels slow	Feels instant!

Optimistic delete example:

```

async function handleDelete(userId) {
// Step 1: Save old state in case we need to roll back
const previousUsers = users;

// Step 2: Update UI IMMEDIATELY (optimistic)
setUsers(prev => prev.filter(u => u.id !== userId));

// Step 3: Try the actual server request
try {
await fetch(`/api/users/${userId}`, { method: 'DELETE' });
// Success! UI is already correct – nothing to do
} catch (error) {

```

```
// Step 4: Server failed – roll back to old state
setUsers(previousUsers);
alert('Delete failed. Please try again.');
```

11. Deleting Data via DELETE HTTP Requests

A **DELETE request** tells the server to **remove a specific item**. You usually pass the item's ID in the URL. DELETE requests typically don't have a request body.

DELETE request example:

```
async function deleteUser(userId) {
  const response = await fetch(`https://api.example.com/users/${userId}`, {
    method: 'DELETE', // Just set the method – no body needed
  });

  if (!response.ok) {
    throw new Error('Delete failed. Status: ' + response.status);
  }
  // If we get here, delete was successful!
}

// In your component:
async function handleDelete(id) {
  try {
    await deleteUser(id);
    setUsers(prev => prev.filter(u => u.id !== id)); // Remove from UI
  } catch (err) {
    setError('Could not delete user.');
```

Quick Reference — HTTP Requests Cheat Sheet

Task	Code Pattern
GET data on load	<code>useEffect(() => { fetch(url).then(r=>r.json()).then(setData) }, [])</code>
async/await fetch	<code>const res = await fetch(url); const data = await res.json();</code>
Check for errors	<code>if (!res.ok) throw new Error('Failed: ' + res.status);</code>
Show loading spinner	<code>const [isLoading, setIsLoading] = useState(false)</code>
Show error message	<code>const [error, setError] = useState(null)</code>
POST new data	<code>fetch(url, { method:'POST', headers:{...}, body:JSON.stringify(data) })</code>

DELETE an item	<code>fetch(`/api/items/\${id}`, { method: 'DELETE' })</code>
Optimistic update	Update state first, send request, rollback on catch
Extract fetch logic	Move all <code>fetch()</code> calls into <code>http.js</code> , import functions
Transform data	<code>data.map(item => ({ cleanKey: item.uglyKey }))</code>